

Behavioral Equivalence Cannot See Dormant Logic: A Pre-Registered Negative Result from a Five-Arm Comparison of PLC Command Gates on a Simulated PLC

WACE Technical Report TR-2026-01 (v1.0, released 2026-09-21)

Dongkeol Bang — WACE Inc., Hwaseong, Republic of Korea (bangdk@wace.me)

Released 2026-09-21 as a defensive publication (the company has decided not to file a patent on this work). Numbers are from official run 6 of the experiment (repository commit 3e7351b, adopted at 65528cc).

Abstract

Command gates that pre-execute supervisory writes on a replica of the PLC program before they reach the controller are a known approach [1]. We ask whether adding a behavioral-equivalence check — confirming, from observed transitions, that the replica still matches the plant and blocking as undecidable otherwise — changes what the gate catches. In a pre-registered five-arm comparison on a simulated PLC (three self-authored ladder programs, 869 labelled commands, frozen catalogue and rule hashes, six official runs with all commits and records published), the arm with the equivalence check and the arm without it let through exactly the same 23 of 280 hazardous commands issued while the replica was stale (8.2%), while the check raised undecidable verdicts under mismatch from 5.7% to 7.0% (38.2% under match in both arms) and blocked 16 of 25 rows under sub-second input changes. Four of the eight person-exposure false passes were motor-start commands issued after an interlock had been removed: a dormant logic change moves no tag until the start command arrives, so an observation-based freshness check never enters suspicion, and the permissive it evaluates is a replica value synchronised at least one second earlier. The structural remedy is a static program-signature check ahead of any behavioral check; on protocols without a program signature (Modbus) the gate cannot see this class of change. The same fail-closed gate blocked 2 of 94 commands that move the plant toward a safe state. Results are from a simulated PLC on one machine; no physical PLC comparison and no externally authored program are included. Our contribution is the evaluation discipline — pre-registration, run invalidation on judgement instability, no post-hoc relabelling, published denominators and excluded rows — shown alongside the numbers it prevented from being reported.

1. Question

When supervisory systems (AI planners, MES, HMI) write directly to a PLC, a wrong command must be stopped in front of the equipment. Gates that hold a replica of the PLC program, run a candidate write a few scans ahead on that replica and block on an invariant violation are public prior art [1]. The premise is that the replica matches the plant. Plant programs drift — a maintenance bypass that is never restored, an online edit — and the replica then keeps judging on stale logic. A natural addition is a **behavioral-equivalence check**: confirm from observed transitions that the replica still matches the plant and, if it cannot be confirmed, block the write as undecidable. Our question is narrow: **does adding that check reduce hazardous false passes?** In this command set the answer is no, and the reason lies in the principle of the check itself.

Static pre-load verification of PLC code [3], the “ladder logic bomb” — logic that stays dormant until a trigger — and its proposed countermeasure of downloading the running logic and diffing it against a golden copy [4], and twin-based detection that lists “the twin models the correct behaviour” as its main limitation [5] are the closest prior work we could confirm in a first-pass survey of 33 sources; we found no controlled comparison that measures the marginal effect of an observation-based freshness check on a command gate.

2. Threat model — two adversaries that must not be conflated

(a) **An untrusted supervisory writer** — the AI planner, MES or HMI issuing the command. This is what the gate exists for, and every false-pass and false-block figure in this report is measured against this adversary. (b) **PLC program drift of any origin** — maintenance not restored, online edits, malicious change — leaving the plant program different from the replica. This is not something the gate blocks; it is a *condition* under which the gate must decide whether its own replica can be trusted. An adversary who can write the PLC program cannot be stopped by any supervisory gate; that belongs to PLC write protection and change management (an IEC 62443 zone boundary). The contribution of this report is exactly: trying to detect (b) from behaviour is blind to dormant changes, so a static integrity check must come first.

3. Experimental design

Gate under test. An external device between the supervisory writer and the PLC. It synchronises the plant PLC’s state (I/O, internal bits, timer elapsed values) into its replica, executes the candidate write on the replica ahead of the plant, and blocks on an invariant violation. It holds no physical-process model, forwards no frames on the PLC’s behalf, and proves by hash that judging leaves no side effect on the replica.

Five arms (Table 1) add one component at a time. ARM3→ARM4 isolates the effect of internal-state synchronisation; ARM4→ARM5 isolates the effect of the behavioral-equivalence check.

Command set. 869 commands over three self-authored ladder programs (conveyor interlock, emergency stop, batch; 3-19 rungs each). Each command carries a replica-matches/replica-stale condition and a hazard-class label (person, product, record, kpi-only) assigned **before** any run; the hashes of the catalogue, rule set and label file were frozen. No customer program was used.

Pre-registered rules. (1) An acceptance line (any block of a command that moves the plant toward a safe state for a reason other than replica mismatch disqualifies the run) and expected directions are written before execution. (2) No unchanged re-run; at most three attempts in total. (3) If the three judging passes disagree on even one row, the run is invalid. (4) A row may be removed from the measured set only if the diagnosis predates the result, a mechanism is derived, and the exclusion is direction-neutral; inclusion and exclusion results are published side by side. (5) Numerators, denominators and excluded row IDs are published; what was not measured is never reported as zero. (6) Per-class counts are reported only as produced by one harness procedure — no hand tallies.

Table 1. Five-arm results (run 6; simulated PLC, three self-authored ladders, one machine)

Arm	Configuration	Hazardous cmds reaching PLC (of 284)	False pass (replica stale)	False block (replica matches, of 309)	Undecidable (stale / matches)	Safe-direction cmds blocked (disqualifying / baseline, of 94)	p95 ms
ARM1	No gate	284	246/284 (0.866)	0	—	—	1.0
ARM2	Static address/value rules only	133	95/284 (0.335)	128/309 (0.414)	0 / 0	47 / 0	1.9
ARM3	Look-ahead execution, output levels synchronised only	66	32 (0.119)	13/309 (0.042)	0.060 / 0.386	0 / 2	36.4
ARM4	Look-ahead execution, internal state synchronised	57	23/280 (0.082)	6/309 (0.019)	0.057 / 0.382	0 / 2	36.4
ARM5	ARM4 + behavioral- equivalence check	57	23/280 (0.082)	6/309 (0.019)	0.070 / 0.382	0 / 2	35.5

The denominator 280 is 284 hazardous commands minus 4 rows whose loaded state already violated a rule and was therefore not a judging target; the four row IDs are in the result file. ARM3’s false-pass denominator is not in its result file, so only the rate is given. Latency is from one 16-core laptop.

4. Results

The equivalence check’s marginal detection gain is zero. ARM4 and ARM5 false passes are identical at 23 of 280. What the check adds is cost: undecidable verdicts under replica mismatch rise from 5.7% to 7.0% (38.2% under match in both arms), and when plant inputs are held for only 0.07-0.3 s, 16 of the 25 rows staged on the live-running plant are blocked as “replica freshness unconfirmed” (none under the adopted 1.2 s hold).

The 23 false passes by class. person 8, product 6, record 5, kpi-only 4. The eight person-exposure rows split in two. (i) **Four rule-shape rows** — writes that passed while a horn duration or an arm dwell had been changed. The invariant was written as an *event* (edge), so the gate could not know an event that happened before synchronisation; rewriting the same fact as a PLC timer *level* is where these close (to be confirmed at zero in run 7 — a prediction, not a result). (ii) **Four unobserved-program-change**

rows — motor-start commands that passed while an interlock had been removed. These do not close by rule changes (Section 5).

Stop-command blocking, in the same paragraph. Of 94 commands that move the plant toward a safe state, 2 were blocked. Both are rows whose loaded state already violated a rule; zero stop commands were blocked for a disqualifying reason (three judging passes identical). This figure does not mean “the gate never blocks a stop” — the reason is merely exempt from disqualification, and the verdict does not consider whether a command reduces the violation. Hence category safety stops are wired around the gate as a deployment requirement (Section 8).

Hazards confirmed unblockable: 3 kinds, 34 test commands. Found by sweeping the 19 writable antecedent positions of the invariants and 2 command bits. Two are operations the ladder legitimately offers (remote reset, remote acknowledge), where the gate cannot distinguish a 1 written by the supervisory system from a 1 pressed by an operator. One is a 32-bit setpoint written as two words, where a single applied word leaves a value nobody commanded (of 30 legitimate changes, 17 were not applied and 4 were half-applied). Other positions have not been swept.

Run ledger. Six official attempts: five valid, one invalid (Table 2). Invalid runs consume a number. The false-pass numerator 23 is unchanged from run 2 through run 6, the disqualification metric is zero from run 2 on, and from run 1 to run 2 the false-pass rate moved *against* us (0.078→0.082). The gate and runtime sources are identical between run 1 and run 6 (tree-hash changes are test fixtures only), and the command-catalogue hash is the same across all six. Marking four rows as unmeasured was a pre-registration deviation introduced after run 4 was invalidated; its reason (rows that could only be staged by a method the product does not have — loading onto a halted plant — with the same diagnosis already in the run-1 record) and time are recorded, and the inclusion (runs 2-3) and exclusion (runs 5-6) versions are published together. The exclusion moved one number: baseline stop blocks 4→2. False-block rate and undecidable rate under match also changed from run 5 (0.032→0.019, 0.387→0.382); their mechanism awaits a row-level comparison, so only the values are stated here.

Table 2. Run ledger

Run	Commit	Valid?	Harness change	ARM5 false pass	Disqualifying / baseline stop blocks	Unmeasured rows
1	140b68d	valid · acceptance line [FAIL] · not adopted	original staging (halted-plant load)	22 (0.078)	2 / 2	0
2	71f890e	valid · superseded	live staging of 25 rows	23 (0.082)	0 / 4	0
3	5f20aa6	valid · superseded (loaded machine)	provenance guard fix	23 (0.082)	0 / 4	0
4	—	invalid (three judging passes disagreed on one row)	as runs 2-3	—	—	—
5	2c5c812	valid · superseded	4 rows unmeasured · denominators published	23 (0.082)	0 / 2	4
6	3e7351b	valid · adopted	denominator carried into result file	23/280 (0.082)	0 / 2	4

5. Mechanism — why the equivalence check cannot see a removed interlock

The behavioral-equivalence check confirms replica-plant agreement from **observed transitions**. A program with a removed interlock moves **no tag** until a start command arrives. With no movement the check never enters suspicion, and when the command arrives the interlock value it evaluates is the replica’s value, synchronised at least one second earlier, which reads “interlock healthy”. A gate that reads permissives from a replica is a PLC reading permissives from a stale buffer. This is not a defect of this implementation but the boundary of observation-based freshness checks as a class: a change to a rung that has not executed does not appear in behaviour.

In the same experiment, 20 of the 30 invariant clauses carried an event (edge) in their antecedent or consequent, in three classes. **A** (8): the judged write itself is the event — snapshot judging works. **B** (3): the event precedes synchronisation but a PLC timer remembers it — rewritable as a level. **C** (6-7): the PLC does not remember it either — undecidable under any history

window; the clause is refused at load and listed by name under “hazards we cannot block”. The design rule becomes: *an event the PLC does not remember cannot be used in an invariant*. Both stop blocks were class B; the four interlock-removal rows were class A — the latter is not a clause problem.

The closing means lie outside behavioral checking — and this direction is not a new recommendation. Static comparison [4] and pre-load static verification [3] already point the same way; what this report adds is a measurement, in this class of gate, of why it is necessary. (1) Static integrity: read the program signature (checksum) the PLC exposes, or upload the program and hash it, and on any difference from the commissioning value enter a “program changed” state that blocks every energising write. The judging order becomes static signature → allow-list → behavioral equivalence → rules. (2) For start/enable commands, read the permissive tags live from the PLC at judging time. Both are design conclusions, not yet measured results. **On protocols with no program signature (Modbus), (1) does not exist**, the gate cannot see this class of change, and PLC write protection plus change management must be in place.

6. What the discipline caught — numbers that would have gone out, and the numbers reported

Rule	Event caught	Would have gone out	Reported
Three judging passes disagree → run invalid	Run 4, one row (the gate correctly suspected a halted plant)	Run 4 adopted as “valid”	Run 4 invalid; four rows marked unmeasured after root cause
Pre-registered acceptance line (stop blocks = 0)	Run 1: two stop commands blocked	“227 blocked” alone	Run 1 not adopted; harness cause fixed before re-run
Per-class counts only from one harness procedure	Lead’s hand estimate: person 4	person 4	person 8
Numerators, denominators and excluded IDs published	Run 5 result file lacked the false-pass denominator	“8.2%” alone	23/280 plus four excluded IDs
No post-hoc relabelling	Registering the four interlock rows as “unblockable” would cut the numerator 22→18	18	Kept in the numerator, published as a separate item
Asymmetric exclusion: three conditions plus two-way publication	Excluding four rows also removed the basis of the “zero stop blocks” claim (1 favourable, 7 unfavourable)	Exclusion effect only	Two-way table plus “this exclusion turns an invalid run into a valid one” stated
Harness provenance guard	Run 3 provenance-guard defect	Result file with wrong provenance	Fixed and re-run

None of these principles is new — researcher degrees of freedom and the 21-word disclosure statement [6][7], pre-coding of outcome variables and the garden of forking paths [8], catalogued benchmarking flaws [9], and the pitfalls of security evaluation (lab-only evaluation, label inaccuracy, inappropriate baselines) [10]; our two-way inclusion/exclusion table is requirement 5 of [6] as written. What we add is machine enforcement (a run is refused when the relaxation set intersects the disqualification-contributing rows; a record is not produced unless the headline figure rebuilds from its published components) and the target (a safety gate on a frozen catalogue with a timing-sensitive harness).

7. Threats to validity

- **Construct.** Hazard-class labels were assigned before any run but by the team that built the gate; there is no independent-labeller agreement. The “3 kinds we confirmed unblockable” and the “6 hazards we deliberately let pass” are listed with class and reason in one table so that neither reads as denominator manipulation.
- **Internal.** Non-physical conditions created by the harness (loading onto a halted plant) were separated as four unmeasured rows with both inclusion and exclusion published. Run 3 was on a loaded machine; its latency is not used.
- **External.** Simulated PLC assembled from product components; three self-authored ladders; one machine; zero externally authored programs. The class in which an equivalence check should matter (forced bits; same program, values overwritten) is absent from this set, so “zero gain” is bounded to this set.
- **Scope of novelty.** In a first-pass survey of 33 sources we found no controlled comparison of the marginal effect of an observation-based freshness check; not finding one is not evidence that none exists.
- **Conclusion.** Exact counts on a fixed catalogue; no statistical test. Three identical judging passes are a reproduction on the same machine and tree, not statistical replication.
- **Generative-AI involvement.** A substantial part of the source code, experiment harness and design documents was written by an LLM coding agent under the author’s direction, and part of the independent judging and review of the experiment rules was carried out by agents of the same model. Approval of the design, fixing of the pre-registration, run validity

decisions, interpretation and every claim in this report were verified by and are the responsibility of the human author.
“Independent judgement” here means agent runs that did not read each other’s output, not human reviewers.

8. Deployment requirements and next steps

Requirements outside the gate. (1) Stop and de-energise actions are wired around the gate — the gate blocks a command whose loaded state already violates a rule even if that command would halt the equipment. (2) Remote-reset and remote-acknowledge bits are removed from supervisory authority. (3) 32-bit setpoints are written with a multi-register write (FC16). (4) When the PLC scan halts (STOP/PROGRAM mode, watchdog halt, online-edit pause) supervisory commands are blocked by design — a halted line whose PLC still scans is not blocked. Timer and counter internals must be exposed to communication, and what counts as a violation must be written for that plant.

Next experiments. (1) One physical PLC (the 25 live-staged rows, the 4 interlock-removal rows, the 94 stop commands, forced bits). (2) The forced-bit / value-overwrite scenario — the only class in which an equivalence check should matter. (3) Run 7 with class-B clauses rewritten as levels, static signature first and live permissive reads — expected directions (rule-shape 4 → 0, interlock-removal 4 → unchanged) registered before execution. (4) At least one externally authored ladder. (5) Measurement of the suspicion-recovery interval (lower bound 8 s, no upper bound).

Known limitations (to be mirrored in the repository’s security notes at release). Person-exposure false passes remain at 8 in run 6; 4 of them do not close by rule changes; on Modbus the gate cannot see dormant logic changes; a sub-second input change can hold the gate in suspicion with no time-based release.

References (sources whose text was opened and checked; verification levels are recorded in the survey file)

- [1] US 12,238,120 B1, “Embedded intrusion prevention system for industrial controllers”, University of Alabama in Huntsville (inventors A. W. Werth, T. H. Morris), priority 2020-12-10, granted 2025-02-25.
- [2] KR 10-2004456 B1, “Apparatus and method for converting a PLC control program into structured data”, UDMTEK Co., Ltd., priority 2016-08-12, granted 2019-07-30. (The gate in this report is designed not to practise the elements of its claims.)
- [3] S. McLaughlin, S. Zonouz, D. Pohly, P. McDaniel, “A Trusted Safety Verifier for Process Controller Code”, NDSS 2014, DOI 10.14722/ndss.2014.23043.
- [4] N. Govil, A. Agrawal, N. O. Tippenhauer, “On Ladder Logic Bombs in Industrial Control Systems”, ESORICS 2017 Workshops (CyberICPS 2017), LNCS, Springer, pp. 110-126, DOI 10.1007/978-3-319-72817-9_8; arXiv:1702.05241.
- [5] M. Eckhart, A. Ekelhart, “A Specification-based State Replication Approach for Digital Twins”, ACM CPS-SPC ’18, pp. 36-47, DOI 10.1145/3264888.3264892.
- [6] J. P. Simmons, L. D. Nelson, U. Simonsohn, “False-Positive Psychology: Undisclosed Flexibility in Data Collection and Analysis Allows Presenting Anything as Significant”, Psychological Science 22(11), 2011, pp. 1359-1366, DOI 10.1177/0956797611417632.
- [7] J. P. Simmons, L. D. Nelson, U. Simonsohn, “A 21 Word Solution”, Dialogue (SPSP newsletter) 26(2), Fall 2012.
- [8] A. Gelman, E. Loken, “The garden of forking paths: Why multiple comparisons can be a problem, even when there is no ‘fishing expedition’ or ‘p-hacking’ and the research hypothesis was posited ahead of time”, unpublished manuscript, 2013-11-14.
- [9] E. van der Kouwe, G. Heiser, D. Andriess, H. Bos, C. Giuffrida, “SoK: Benchmarking Flaws in Systems Security”, IEEE EuroS&P 2019, pp. 310-325, DOI 10.1109/EuroSP.2019.00031.
- [10] D. Arp, E. Quiring, F. Pendlebury, A. Warnecke, F. Pierazzi, C. Wressnegger, L. Cavallaro, K. Rieck, “Dos and Don’ts of Machine Learning in Computer Security”, USENIX Security 2022.